

1.2. Decisiones de diseño justificadas

1.2.1. Cuadro Comparativo y justificaciones de diseño

1.2.1.1. Decisión 1 (Front-End): Maquetación adaptativa

- **Alternativas comparadas:** CSS Flexbox/Grid frente a maquetación tradicional mediante bloques simples (display: block) con posicionamiento flotante.
- **Criterios de evaluación:**

1. **Usabilidad:** Facilidad para reorganizar el catálogo de arreglos y el formulario de pedidos en layouts móviles y de escritorio.

2. **Mantenibilidad:** Limpieza del código CSS y nivel de esfuerzo para ajustar la interfaz.

- **Matriz de comparación:**

Criterio	CSS Flexbox/Grid	Bloques simples
Usabilidad	Alta: Permite cambiar fácilmente de 1 columna móviles a un sistema multidireccional en escritorios mediante @media queries.	Baja: Requiere trucos de reseteo (clears), rompe el flujo ante cambios de resolución y desborda en smartphones.
Mantenibilidad	Alta: Control centralizado con propiedades nativas como gap, flex-direction y grid-template-columns.	Baja: Código CSS extenso, repetitivo y frágil ante cambios de diseño.

- **Justificación de la elección:** Se selecciono CSS Flexbox y Grid. Permite estructurar de forma limpia el catálogo de arreglos y ajustar dinámicamente el formulario de pedidos de "RitmoDeDetalles" en pantallas reducidas (≤ 768 px), reduciendo errores visuales.

1.2.1.2. Decisión 2 (Back-End): Gestión y almacenamiento de datos

- **Alternativas comparadas:** Almacenamiento volátil en memoria JSON en Node.js vs. Sistema de Gestión de Bases de Datos (MySQL).
- **Criterios de evaluación:**

1. **Tiempo de ejecución:** Complejidad de instalación e inicialización del servidor local.

2. **Complejidad:** Nivel de sobrecarga técnica y necesidad de dependencias externas de pago o complejas.

- **Matriz de comparación:**

Criterio	Memoria/Archivo JSON	Bloques simples
Tiempo de ejecución	Óptima: Inicialización inmediata mediante el entorno Node.js nativo y Express.js sin requerir software de terceros	Alto: Exige instalar, configurar y mantener corriendo servicios como MySQL Server de forma independiente.
Complejidad	Baja: Se maneja con estructuras nativas de JS (arreglos/objetos) de forma gratuita.	Alta: Exige clientes de conexión (mysql2), scripts migración SQL y manejo de esquemas rígidos.

- **Justificación de la elección:** Se selecciona almacenamiento en memoria o archivo JSON local mediante Express.js. Garantiza un entorno de ejecución ágil, completamente gratuito, portable y ajustado a la naturaleza del prototipo sin añadir sobrecarga de bases de datos.

1.2.2. Declaración de supuestos

- **Supuesto operativo de cliente (dispositivo móvil):** Se asume que la mayoría de los clientes de "RitmoDeDetalles" accederán a la página web desde sus teléfonos inteligentes (smartphones) mientras navegan en sus redes sociales, por lo que la prioridad del diseño front-end debe centrarse en una vista móvil fluida y limpia (≤ 768 px).

- **Supuesto operativo de administración (entorno de escritorio):** Se asume que la administradora de la floristería revisará la lista de pedidos consolidados y gestionará los estados (Pendiente, En Preparación, Entregado) desde una computadora portátil o laptop en su taller artesanal, por lo que el panel de administración debe priorizar la visualización en tablas anchas con componentes Bootstrap.

1.2.3. Aplicación de conocimientos de programación

En el desarrollo de este proyecto se aplican directamente conceptos fundamentales de lógica de programación y organización de datos aprendidos en asignaturas previas:

- **Estructuras de datos y modelado de datos:** La lógica para gestionar un pedido de la floristería aplica el concepto de registros/diccionarios de Python traducido a objetos de JavaScript (`{id, cliente, telefono, tipoArreglo, fechaEntrega, estado}`), mientras que la lista de pedidos equivale a una lista en Python manipulada en JavaScript mediante un arreglo de objetos (`const pedidos = []`).
- **Estructuras de control y lógica:** Las condicionales aprendidas (`if/else`) se emplean para validar que los campos del formulario no estén vacíos antes de enviarlos y para asignar las respuestas de estado HTTP en el servidor (201 si se guarda el pedido, 400 si faltan datos requeridos).
- **Clarificación de alcance técnico:** Se deja constancia de que los principios lógicos aprendidos en Python sirvieron como base de razonamiento algorítmico, pero la aplicación web se desarrollará sobre JavaScript (Front-end) y Node.js/Express (Back-end) sin necesidad de ejecutar intérpretes de Python en la solución.